O menor autômato

Minimização, gramáticas e a hierarquia

Encontro 6 | 2 de setembro

Linguagens Formais e Compiladores

Uma coisa que aconteceu na oficina

Quando vocês projetaram autômatos para o mesmo enunciado,
saíram desenhos **diferentes**.

Uma coisa que aconteceu na oficina

Quando vocês projetaram autômatos para o mesmo enunciado,
saíram desenhos **diferentes**.

Alguns com mais estados, outros com menos. E todos funcionavam.

Uma coisa que aconteceu na oficina

Quando vocês projetaram autômatos para o mesmo enunciado,
saíram desenhos **diferentes**.

Alguns com mais estados, outros com menos. E todos funcionavam.

Existe um que seja o certo?

A pergunta de hoje

Dada uma linguagem, existe um *melhor* autômato para ela?

A pergunta de hoje

Dada uma linguagem, existe um *melhor* autômato para ela?

Existe. É o menor, e ele é único.

A pergunta de hoje

Dada uma linguagem, existe um *melhor* autômato para ela?

Existe. É o menor, e ele é único.

Hoje vamos aprender a encontrá-lo.

1

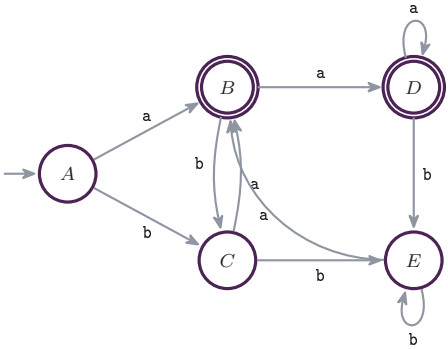
Três autômatos, uma linguagem

A linguagem que vamos usar

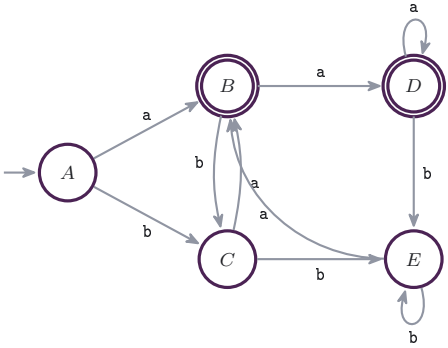
Palavras que terminam com a.

aceita	a, ba, aa, abba
rejeita	b, ab, bb, abab

Autômato 1: cinco estados

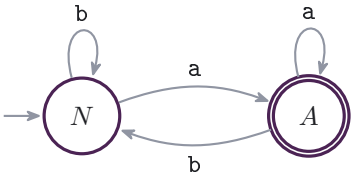


Autômato 1: cinco estados

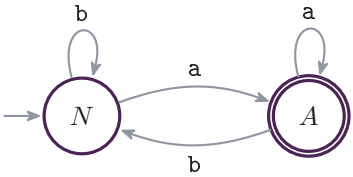


Funciona. Testei todas as 1.023 palavras de até 9 símbolos.

Autômato 2: dois estados

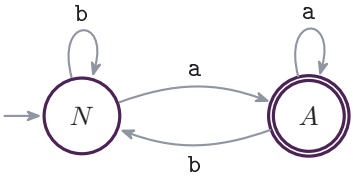


Autômato 2: dois estados



Também funciona. As mesmas 1.023 palavras, o mesmo resultado.

Autômato 2: dois estados



Também funciona. As mesmas 1.023 palavras, o mesmo resultado.

Os dois estão corretos. O segundo usa menos da metade.

O que sobrava no primeiro

Três estados a mais, e a linguagem é a mesma.

O que sobrava no primeiro

Três estados a mais, e a linguagem é a mesma.

Onde estava o desperdício?

O que sobrava no primeiro

Três estados a mais, e a linguagem é a mesma.

Onde estava o desperdício?

Existem dois tipos, e o resto da primeira metade da aula é sobre eles.

2

Desperdício tipo 1: estado inalcançável

Um estado onde ninguém chega



Um estado onde ninguém chega



Comece em q_0 e siga as flechas. Você chega em q_1, e volta.
Não existe caminho para q_2 nem para q_3.

Um estado onde ninguém chega



Comece em q_0 e siga as flechas. Você chega em q_1, e volta.
Não existe caminho para q_2 nem para q_3.

Apague os dois. Nenhuma palavra passava por lá.

Como achar todos

Marque o estado inicial. Depois marque tudo que tem flecha vindo de um marcado. Repita até parar de marcar.

Como achar todos

Marque o estado inicial. Depois marque tudo que tem flecha vindo de um marcado. Repita até parar de marcar.

O que ficou sem marca é inalcançável.

Como achar todos

Marque o estado inicial. Depois marque tudo que tem flecha vindo de um marcado. Repita até parar de marcar.

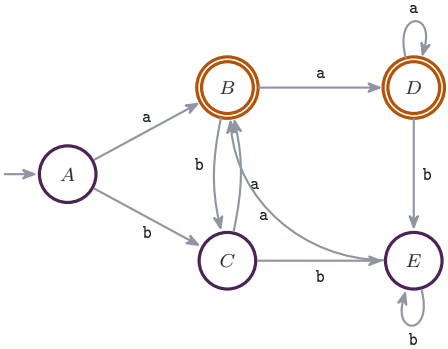
O que ficou sem marca é inalcançável.

No autômato de cinco estados que vamos minimizar, todos são alcançáveis. O desperdício ali é do outro tipo.

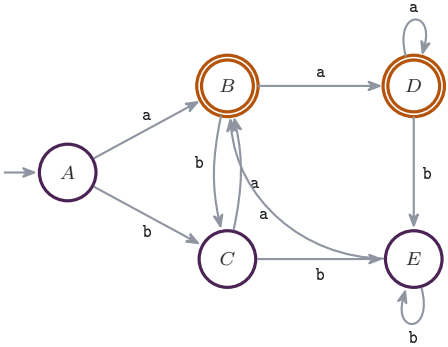
3

Desperdício tipo 2: estados equivalentes

Olhem B e D

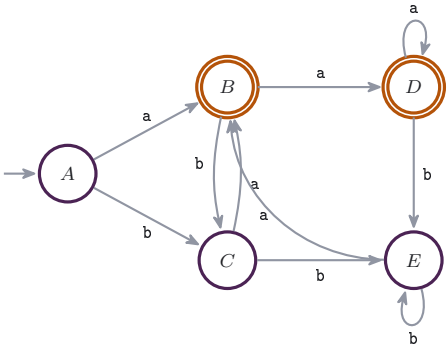


Olhem B e D



Os dois são finais, e os dois têm uma flecha a e uma flecha b.

Olhem B e D



Os dois são finais, e os dois têm uma flecha a e uma flecha b.

Será que dá para juntar os dois num só?

Vamos testar com a palavra `bab`

Coloque uma ficha em B e outra em D, e leia `bab` nas duas.

Vamos testar com a palavra `bab`

Coloque uma ficha em B e outra em D, e leia `bab` nas duas.

começando em B: $\quad B \to C \to B \to C$

Vamos testar com a palavra `bab`

Coloque uma ficha em B e outra em D, e leia `bab` nas duas.

começando em B: $B \to C \to B \to C$

começando em D: $D \to E \to B \to C$

Vamos testar com a palavra `bab`

Coloque uma ficha em B e outra em D, e leia `bab` nas duas.

começando em B: $B \to C \to B \to C$

começando em D: $D \to E \to B \to C$

Os caminhos são **diferentes**. E os dois terminam em C, que não é final.

Vamos testar com a palavra `bab`

Coloque uma ficha em B e outra em D, e leia `bab` nas duas.

começando em B: $B \to C \to B \to C$

começando em D: $D \to E \to B \to C$

Os caminhos são **diferentes**. E os dois terminam em C, que não é final.

As duas fichas dão a mesma resposta: rejeita.

E com as outras palavras?

Testei as 512 palavras de até 8 símbolos, partindo de B e de D.

E com as outras palavras?

Testei as 512 palavras de até 8 símbolos, partindo de B e de D.

Zero divergências.

E com as outras palavras?

Testei as 512 palavras de até 8 símbolos, partindo de B e de D.

Zero divergências.

Não existe palavra que B aceite e D rejeite, nem o contrário.

A definição

DEFINIÇÃO • ESTADOS EQUIVALENTES

Dois estados são equivalentes quando, partindo de qualquer um deles, **as mesmas palavras** levam à aceitação.

A definição

DEFINIÇÃO • ESTADOS EQUIVALENTES

Dois estados são equivalentes quando, partindo de qualquer um deles, **as mesmas palavras** levam à aceitação.

Colocado de outro jeito: se você fosse largado num dos dois sem saber em qual, isso não mudaria nenhuma resposta daí para a frente.

A definição

DEFINIÇÃO • ESTADOS EQUIVALENTES

Dois estados são equivalentes quando, partindo de qualquer um deles, **as mesmas palavras** levam à aceitação.

Colocado de outro jeito: se você fosse largado num dos dois sem saber em qual, isso não mudaria nenhuma resposta daí para a frente.

Estados equivalentes podem virar um só.

O problema de testar assim

Para ter certeza de que B e D são equivalentes,
eu testei 512 palavras. E isso não prova nada:
faltam as de 9 símbolos, de 10, de mil.

O problema de testar assim

Para ter certeza de que B e D são equivalentes,
eu testei 512 palavras. E isso não prova nada:
faltam as de 9 símbolos, de 10, de mil.

Precisamos de um jeito de decidir sem testar palavra nenhuma.

O problema de testar assim

Para ter certeza de que B e D são equivalentes,
eu testei 512 palavras. E isso não prova nada:
faltam as de 9 símbolos, de 10, de mil.

Precisamos de um jeito de decidir sem testar palavra nenhuma.

É o algoritmo do próximo bloco.

4
O algoritmo

A ideia

Em vez de procurar quem é equivalente, procure quem **não é**.

A ideia

Em vez de procurar quem é equivalente, procure quem **não é**.

Comece com a separação óbvia:
um estado final nunca é equivalente a um não final.

A ideia

Em vez de procurar quem é equivalente, procure quem **não é**.

Comece com a separação óbvia:
um estado final nunca é equivalente a um não final.

Porque a palavra vazia já os distingue: parando ali, um aceita e o outro não.

A tabela do autômato de cinco estados

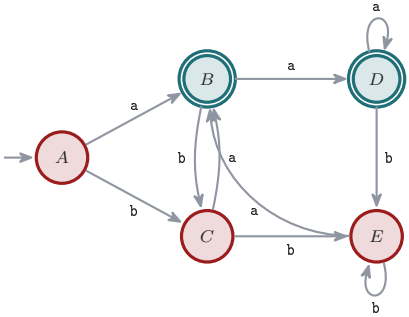
estado	**lendo** `a`	**lendo** `b`	**final?**
A (inicial)	B	C	não
B	D	C	**sim**
C	B	E	não
D	D	E	**sim**
E	B	E	não

A tabela do autômato de cinco estados

estado	**lendo** `a`	**lendo** `b`	**final?**
A (inicial)	B	C	não
B	D	C	**sim**
C	B	E	não
D	D	E	**sim**
E	B	E	não

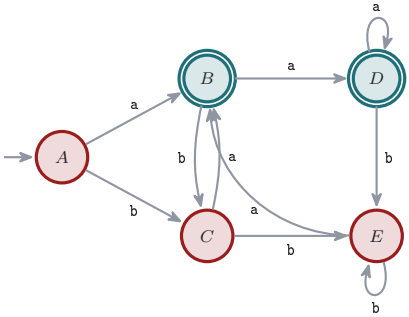
É o mesmo autômato do desenho, escrito em tabela.
Fica mais fácil de percorrer com o dedo.

Passo 0: separe finais de não finais



Verde: os finais. **Vermelho**: os outros três.

Passo 0: separe finais de não finais



Verde: os finais. **Vermelho**: os outros três.

A palavra vazia já separa os dois grupos: parando ali, o verde aceita e o vermelho não.

A pergunta que refina

Dentro de um mesmo grupo, os estados vão para os *mesmos grupos*?

A pergunta que refina

Dentro de um mesmo grupo, os estados vão para os *mesmos grupos*?

Se dois estados do mesmo grupo levam a grupos diferentes,
existe uma palavra que os distingue. Eles precisam ser separados.

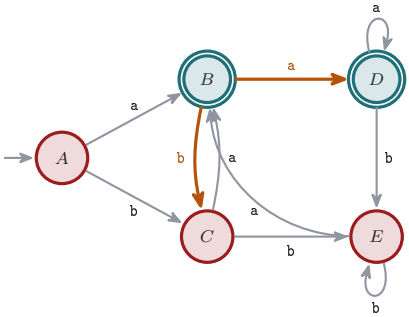
A pergunta que refina

Dentro de um mesmo grupo, os estados vão para os *mesmos grupos*?

Se dois estados do mesmo grupo levam a grupos diferentes,
existe uma palavra que os distingue. Eles precisam ser separados.

Vamos verificar um estado de cada vez, seguindo as flechas.

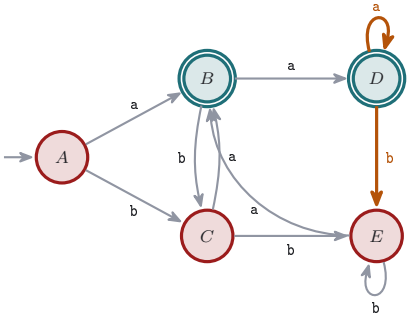
Grupo verde: examinando B



lendo a vai para D que está no **verde**

lendo b vai para C que está no **vermelho**

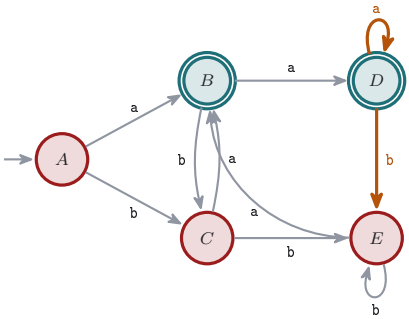
Grupo verde: examinando D



lendo a vai para D que está no **verde**

lendo b vai para E que está no **vermelho**

Grupo verde: examinando D



lendo a vai para D que está no **verde**

lendo b vai para E que está no **vermelho**

B e D vão para os mesmos grupos. Ficam juntos.

Um detalhe importante

Lendo `b`, B foi para C e D foi para E.

Um detalhe importante

Lendo b, B foi para C e D foi para E.

São estados **diferentes**.

Um detalhe importante

Lendo `b`, B foi para C e D foi para E.

São estados **diferentes**.

E isso não importa. O que conta é que os dois estão no mesmo grupo.

Um detalhe importante

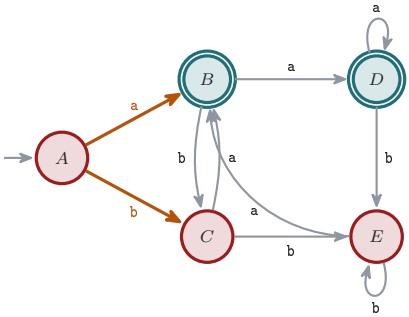
Lendo b, B foi para C e D foi para E.

São estados **diferentes**.

E isso não importa. O que conta é que os dois estão no mesmo grupo.

Se C e E vierem a ser separados num passo seguinte,
aí sim B e D teriam de ser reexaminados.

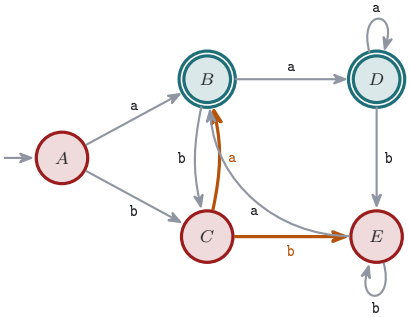
Grupo vermelho: examinando A



lendo a vai para B **verde**

lendo b vai para C **vermelho**

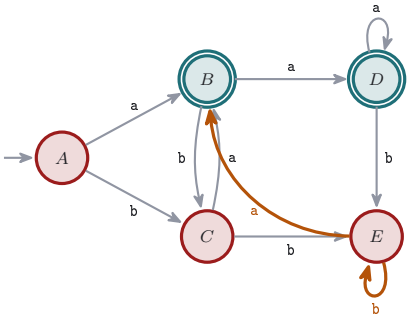
Grupo vermelho: examinando C



lendo a vai para B **verde**

lendo b vai para E **vermelho**

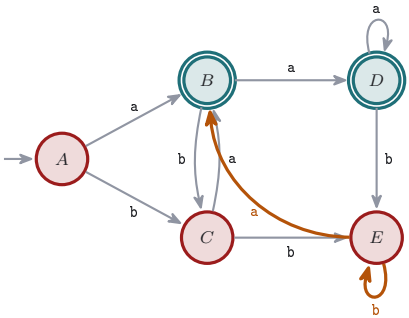
Grupo vermelho: examinando E



lendo a vai para B **verde**

lendo b vai para E **vermelho**

Grupo vermelho: examinando E



lendo a vai para B **verde**

lendo b vai para E **vermelho**

Os três vão para verde e vermelho. Ficam juntos.

Nenhum grupo se dividiu



Nenhum grupo se dividiu



Quando um passo não separa nada, o algoritmo terminou.

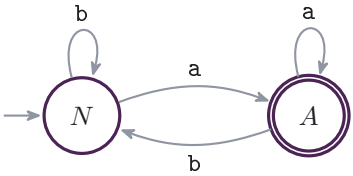
Nenhum grupo se dividiu



Quando um passo não separa nada, o algoritmo terminou.

Cada grupo que sobrou vira um estado do autômato mínimo.

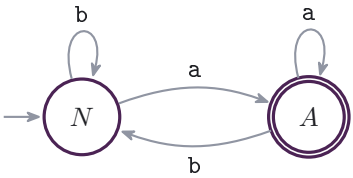
O autômato mínimo



N vem de $\{A, C, E\}$ a última letra não foi a

A vem de $\{B, D\}$ a última letra foi a

O autômato mínimo



N vem de $\{A, C, E\}$ a última letra não foi a

A vem de $\{B, D\}$ a última letra foi a

É o autômato 2, que eu mostrei no começo da aula.

Por que dois estados bastam

Para saber se uma palavra termina com a, o que é preciso lembrar?

Por que dois estados bastam

Para saber se uma palavra termina com a, o que é preciso lembrar?

Só a última letra lida.

Por que dois estados bastam

Para saber se uma palavra termina com a, o que é preciso lembrar?

Só a última letra lida.

Duas respostas possíveis, dois estados.
Os outros três guardavam informação que nunca era consultada.

5
Exercício

Minimizem este

Aceita palavras que terminam com `ab`. Cinco estados.

estado	**lendo** `a`	**lendo** `b`	**final?**
A (inicial)	B	C	não
B	B	E	não
C	D	C	não
D	D	E	não
E	B	C	**sim**

Minimizem este

Aceita palavras que terminam com `ab`. Cinco estados.

estado	**lendo** `a`	**lendo** `b`	**final?**
A (inicial)	B	C	não
B	B	E	não
C	D	C	não
D	D	E	não
E	B	C	**sim**

Separem finais de não finais. Depois vejam para que grupo cada um vai.

A resposta


passo 0 · finais e não finais
E
A
B
C
D
passo 1 · o grupo grande se dividiu
E
B
D
A
C

A resposta



Lendo a, os quatro do grupo grande vão para o mesmo lugar.
Lendo b, B e D vão para E, que é final. A e C não.

A resposta



Lendo a, os quatro do grupo grande vão para o mesmo lugar.
Lendo b, B e D vão para E, que é final. A e C não.

O b separou. Cinco estados viram três.

O que cada grupo lembrava

$\{A, C\}$	ainda não vi a
$\{B, D\}$	vi a
$\{E\}$	vi ab

O que cada grupo lembrava

$\{A, C\}$	ainda não vi a
$\{B, D\}$	vi a
$\{E\}$	vi ab

Três situações, três estados.

O que cada grupo lembrava

$\{A, C\}$	ainda não vi a
$\{B, D\}$	vi a
$\{E\}$	vi ab

Três situações, três estados.

O autômato original tinha duas cópias de cada uma das duas primeiras.

Segundo exercício: sete estados

estado	lendo `a`	lendo `b`	final?
q_0 (inicial)	q_3	q_2	não
q_1	q_5	q_3	**sim**
q_2	q_2	q_0	**sim**
q_3	q_3	q_4	não
q_4	q_5	q_6	**sim**
q_5	q_6	q_1	não
q_6	q_3	q_4	não

Segundo exercício: sete estados

estado	lendo `a`	lendo `b`	final?
q_0 (inicial)	q_3	q_2	não
q_1	q_5	q_3	**sim**
q_2	q_2	q_0	**sim**
q_3	q_3	q_4	não
q_4	q_5	q_6	**sim**
q_5	q_6	q_1	não
q_6	q_3	q_4	não

Este não termina na primeira rodada. Vão até nada mais se dividir.

Rodada 0 e rodada 1

rodada 0 $\{q_1, q_2, q_4\}$ e $\{q_0, q_3, q_5, q_6\}$

Rodada 0 e rodada 1

rodada 0 $\{q_1, q_2, q_4\}$ e $\{q_0, q_3, q_5, q_6\}$

rodada 1 $\{q_1, q_4\}$ $\{q_2\}$ $\{q_0, q_3, q_5, q_6\}$

Rodada 0 e rodada 1

rodada 0 $\{q_1, q_2, q_4\}$ e $\{q_0, q_3, q_5, q_6\}$

rodada 1 $\{q_1, q_4\}$ $\{q_2\}$ $\{q_0, q_3, q_5, q_6\}$

O grupo dos finais se dividiu, e foi o a que separou:

estado	**lendo** a	**destino é**
q_1	q_5	não final
q_4	q_5	não final
q_2	q_2	final

Rodada 0 e rodada 1

rodada 0 $\{q_1, q_2, q_4\}$ e $\{q_0, q_3, q_5, q_6\}$

rodada 1 $\{q_1, q_4\}$ $\{q_2\}$ $\{q_0, q_3, q_5, q_6\}$

O grupo dos finais se dividiu, e foi o a que separou:

estado	**lendo** a	**destino é**
q_1	q_5	não final
q_4	q_5	não final
q_2	q_2	final

Agora existem três grupos. E tudo precisa ser reexaminado.

Rodada 2

Com três grupos, a pergunta ficou mais fina.
Olhem o grupo de quatro, lendo b:

estado	vai para	que agora está no grupo
q_3	q_4	$\{q_1, q_4\}$
q_5	q_1	$\{q_1, q_4\}$
q_6	q_4	$\{q_1, q_4\}$
q_0	q_2	$\{q_2\}$

Rodada 2

Com três grupos, a pergunta ficou mais fina.
Olhem o grupo de quatro, lendo b:

estado	vai para	que agora está no grupo
q_3	q_4	$\{q_1, q_4\}$
q_5	q_1	$\{q_1, q_4\}$
q_6	q_4	$\{q_1, q_4\}$
q_0	q_2	$\{q_2\}$

q_0 se separa dos outros três.

O resultado do segundo exercício

rodada 0	2 grupos
rodada 1	3 grupos
rodada 2	4 grupos
rodada 3	nada mudou. Parou.

O resultado do segundo exercício

rodada 0	2 grupos
rodada 1	3 grupos
rodada 2	4 grupos
rodada 3	nada mudou. Parou.

Sete estados viram quatro.

O resultado do segundo exercício

rodada 0	2 grupos
rodada 1	3 grupos
rodada 2	4 grupos
rodada 3	nada mudou. Parou.

Sete estados viram quatro.

Cada rodada distingue estados por palavras um símbolo mais longas.
A rodada 2 só foi possível porque a rodada 1 criou o grupo $\{q_2\}$.

6
O mínimo é único

Uma garantia

Para cada linguagem regular existe um único autômato mínimo, a menos dos nomes que você dá aos estados.

Uma garantia

Para cada linguagem regular existe um único autômato mínimo, a menos dos nomes que você dá aos estados.

Não importa de qual autômato você partiu,
nem em que ordem juntou os estados. O destino é o mesmo.

Uma garantia

Para cada linguagem regular existe um único autômato mínimo, a menos dos nomes que você dá aos estados.

Não importa de qual autômato você partiu,
nem em que ordem juntou os estados. O destino é o mesmo.

A demonstração usa o teorema de Myhill–Nerode, e não vamos fazê-la aqui.

Verificando

Dois autômatos diferentes para "termina com a":

	antes	**depois de minimizar**
o de cinco estados	5	**2**
outro, de três	3	**2**

Verificando

Dois autômatos diferentes para "termina com a":

	antes	depois de minimizar
o de cinco estados	5	2
outro, de três	3	2

OBSERVE

Os dois mínimos são o **mesmo** autômato. Verificado nas 1.023 palavras de até 9 símbolos.

Para que serve na prática

- **Comparar.** Dois autômatos reconhecem a mesma linguagem? Minimize os dois e veja se dão o mesmo.

Para que serve na prática

- **Comparar.** Dois autômatos reconhecem a mesma linguagem? Minimize os dois e veja se dão o mesmo.
- **Economizar.** A tabela de transições ocupa memória proporcional aos estados.

Para que serve na prática

- **Comparar.** Dois autômatos reconhecem a mesma linguagem? Minimize os dois e veja se dão o mesmo.
- **Economizar.** A tabela de transições ocupa memória proporcional aos estados.
- **Conferir o seu desenho.** Se o seu autômato minimiza para bem menos, você criou estados repetidos sem perceber.

Para que serve na prática

- **Comparar.** Dois autômatos reconhecem a mesma linguagem? Minimize os dois e veja se dão o mesmo.
- **Economizar.** A tabela de transições ocupa memória proporcional aos estados.
- **Conferir o seu desenho.** Se o seu autômato minimiza para bem menos, você criou estados repetidos sem perceber.

Geradores de analisador léxico minimizam antes de emitir a tabela.

7
Gramáticas

Onde vocês estão no compilador



Onde vocês estão no compilador



A primeira fase vocês construíram na E1, a partir de **autômatos**.

Onde vocês estão no compilador



A primeira fase vocês construíram na E1, a partir de **autômatos**.

A segunda fase não usa autômato. Usa gramática.

Assim é que se define a sintaxe de uma linguagem

$$
\begin{aligned}
\texttt{comando} &\rightarrow \texttt{se (expr) comando} \\
&\mid \texttt{enquanto (expr) comando} \\
&\mid \texttt{\{ lista \}} \\
&\mid \texttt{ident = expr ;} \\
\texttt{lista} &\rightarrow \texttt{comando lista} \mid \varepsilon
\end{aligned}
$$

Assim é que se define a sintaxe de uma linguagem

$$
\begin{aligned}
\texttt{comando} &\rightarrow \texttt{se (expr) comando} \\
&\mid \texttt{enquanto (expr) comando} \\
&\mid \texttt{\{ lista \}} \\
&\mid \texttt{ident = expr ;} \\
\texttt{lista} &\rightarrow \texttt{comando lista} \mid \varepsilon
\end{aligned}
$$

Isso é um trecho de gramática, e é assim que todo manual de linguagem descreve o que é um programa válido.

Assim é que se define a sintaxe de uma linguagem

$$
\begin{array}{rcl}
\texttt{comando} & \rightarrow & \texttt{se (expr) comando} \\
& | & \texttt{enquanto (expr) comando} \\
& | & \texttt{\{ lista \}} \\
& | & \texttt{ident = expr ;} \\
\texttt{lista} & \rightarrow & \texttt{comando lista} \mid \varepsilon
\end{array}
$$

Isso é um trecho de gramática, e é assim que todo manual de linguagem descreve o que é um programa válido.

Vocês vão escrever uma dessas na E2.

Por que autômato não serve aqui

$$\texttt{comando} \rightarrow \texttt{se (expr)}\ \texttt{comando}$$

Por que autômato não serve aqui

$$\texttt{comando} \rightarrow \texttt{se (expr)}\ \texttt{comando}$$

Um comando pode conter outro comando, que pode conter outro.
Um se dentro de um enquanto dentro de outro se.

Por que autômato não serve aqui

$$\texttt{comando} \rightarrow \texttt{se (expr)}\ \texttt{comando}$$

Um comando pode conter outro comando, que pode conter outro.
Um se dentro de um enquanto dentro de outro se.

Não existe limite para essa profundidade.

Por que autômato não serve aqui

`comando` → `se ( expr )` `comando`

Um comando pode conter outro comando, que pode conter outro.
Um `se` dentro de um `enquanto` dentro de outro `se`.

Não existe limite para essa profundidade.

E um autômato finito tem, por definição, um número finito de estados.
Ele não consegue lembrar quantas chaves foram abertas.

O plano de hoje

Gramáticas são o assunto da próxima unidade inteira.

O plano de hoje

Gramáticas são o assunto da próxima unidade inteira.

Hoje vemos o caso mais simples delas:
o que equivale exatamente aos autômatos que vocês já dominam.

O plano de hoje

Gramáticas são o assunto da próxima unidade inteira.

Hoje vemos o caso mais simples delas:
o que equivale exatamente aos autômatos que vocês já dominam.

Assim, quando a próxima unidade soltar as amarras,
vocês já saberão ler e escrever uma gramática.

Uma regra é uma troca autorizada

$$S \rightarrow \mathtt{a}S$$

Uma regra é uma troca autorizada

$$S \rightarrow \mathtt{a}S$$

Lê-se: *onde estiver* S*, você pode trocar por* aS.

Uma regra é uma troca autorizada

$$S \rightarrow \mathrm{a}S$$

Lê-se: *onde estiver* S*, você pode trocar por* aS.

A seta não é igualdade nem implicação. É uma permissão de substituição.

Dois tipos de símbolo

terminal	minúscula	a, b
não terminal	maiúscula	S, N, A

Dois tipos de símbolo

terminal	minúscula	a, b
não terminal	maiúscula	S, N, A

terminal	fica na palavra e não muda mais
não terminal	é um **buraco** que ainda precisa ser preenchido

Dois tipos de símbolo

terminal	minúscula	a, b
não terminal	maiúscula	S, N, A

terminal	fica na palavra e não muda mais
não terminal	é um **buraco** que ainda precisa ser preenchido

A palavra está pronta quando não sobra nenhuma maiúscula.

Uma gramática com duas regras

$$S \to \mathtt{a}S \qquad S \to \mathtt{b}$$

Uma gramática com duas regras

$$S \to \mathtt{a}S \qquad S \to \mathtt{b}$$

Duas trocas permitidas. Começa-se sempre pelo S.

Uma gramática com duas regras

$$S \to \mathtt{a}S \qquad S \to \mathtt{b}$$

Duas trocas permitidas. Começa-se sempre pelo S.

A primeira regra deixa um S no fim: dá para usá-la de novo.
A segunda não deixa S nenhum: ela **encerra**.

Derivando

$$S$$

Derivando

$$S \Rightarrow \mathsf{a}S$$

usei $S \to \mathsf{a}S$

Derivando

$$S \Rightarrow \mathtt{a}S \Rightarrow \mathtt{aa}S$$

usei $S \to \mathtt{a}S$ de novo

Derivando

$$S \Rightarrow \mathtt{a}S \Rightarrow \mathtt{aa}S \Rightarrow \mathtt{aab}$$

usei $S \to \mathtt{b}$, que não deixa maiúscula. Acabou.

Derivando

$$S \Rightarrow \mathtt{a}S \Rightarrow \mathtt{aa}S \Rightarrow \mathtt{aab}$$

usei $S \to \mathtt{b}$, que não deixa maiúscula. Acabou.

A palavra gerada foi `aab`.

Para que serve derivar

Gerar palavras parece inútil: ninguém quer que o computador escreva programas ao acaso.

Para que serve derivar

Gerar palavras parece inútil: ninguém quer que o computador escreva programas ao acaso.

Mas a derivação é a *prova* de que a palavra pertence à linguagem.

Para que serve derivar

Gerar palavras parece inútil: ninguém quer que o computador escreva programas ao acaso.

Mas a derivação é a *prova* de que a palavra pertence à linguagem.

O analisador sintático faz o caminho inverso: recebe o programa e tenta reconstruir a derivação que o produziria.

Para que serve derivar

Gerar palavras parece inútil: ninguém quer que o computador escreva programas ao acaso.

Mas a derivação é a *prova* de que a palavra pertence à linguagem.

O analisador sintático faz o caminho inverso: recebe o programa e tenta reconstruir a derivação que o produziria.

Se ele consegue, o programa está sintaticamente correto.
Se não consegue, ele aponta onde falhou: é a mensagem de erro.

Que linguagem essa gramática gera

Usando a primeira regra zero, uma, duas, três vezes...
e depois a segunda para encerrar:

`b, ab, aab, aaab, aaaab, ...`

Que linguagem essa gramática gera

Usando a primeira regra zero, uma, duas, três vezes...
e depois a segunda para encerrar:

`b`, `ab`, `aab`, `aaab`, `aaaab`, ...

Zero ou mais `a`, seguidos de um `b`.

Que linguagem essa gramática gera

Usando a primeira regra zero, uma, duas, três vezes...
e depois a segunda para encerrar:

`b`, `ab`, `aab`, `aaab`, `aaaab`, ...

Zero ou mais `a`, seguidos de um `b`.

Em expressão regular: `a*b`

O que torna uma gramática *regular*

Nas duas regras que usamos, o não terminal aparecia sempre no **fim** do lado direito, ou não aparecia.

O que torna uma gramática *regular*

Nas duas regras que usamos, o não terminal aparecia sempre no **fim** do lado direito, ou não aparecia.

permitido $S \to \mathtt{a}S$ $S \to \mathtt{b}$ $S \to \varepsilon$

proibido $S \to \mathtt{a}S\mathtt{b}$ $S \to SS$

O que torna uma gramática *regular*

Nas duas regras que usamos, o não terminal aparecia sempre no **fim** do lado direito, ou não aparecia.

permitido	$S \to \mathtt{a}S$	$S \to \mathtt{b}$	$S \to \varepsilon$
proibido	$S \to \mathtt{a}S\mathtt{b}$	$S \to SS$	

Essa restrição é o que mantém a gramática dentro das linguagens regulares.

Essa restrição é exatamente um autômato

Numa gramática regular, a derivação sempre tem esta cara:

$$\mathsf{a}\,\mathsf{a}\,\mathsf{b}\,\mathsf{a}\,X$$

Essa restrição é exatamente um autômato

Numa gramática regular, a derivação sempre tem esta cara:

$$\mathtt{a}\,\mathtt{a}\,\mathtt{b}\,\mathtt{a}\,X$$

Letras já decididas à esquerda, e **um** não terminal no fim.

Essa restrição é exatamente um autômato

Numa gramática regular, a derivação sempre tem esta cara:

$$\mathtt{a\,a\,b\,a}\,X$$

Letras já decididas à esquerda, e **um** não terminal no fim.

As letras são o que já foi lido. O não terminal é o *estado atual*.

A correspondência

no autômato	**na derivação**
estar num estado	ter aquele não terminal no fim
ler um símbolo e mudar de estado	aplicar uma regra
estar num estado final	poder aplicar $X \to \varepsilon$ e parar

A correspondência

no autômato	**na derivação**
estar num estado	ter aquele não terminal no fim
ler um símbolo e mudar de estado	aplicar uma regra
estar num estado final	poder aplicar $X \to \varepsilon$ e parar

Gramática regular e autômato finito são a mesma coisa, escrita de dois jeitos.

E se soltarmos a restrição

Se o não terminal pudesse ficar no meio, como em $S \to \mathtt{a}S\mathtt{b}$:

$$\mathtt{a}\,\mathtt{a}\,S\,\mathtt{b}\,\mathtt{b}$$

E se soltarmos a restrição

Se o não terminal pudesse ficar no meio, como em $S \to \mathtt{a}S\mathtt{b}$:

$$\mathtt{a}\,\mathtt{a}\,S\,\mathtt{b}\,\mathtt{b}$$

Agora a palavra cresce dos **dois lados ao mesmo tempo**,
e sempre na mesma quantidade.

E se soltarmos a restrição

Se o não terminal pudesse ficar no meio, como em $S \to \mathtt{a}S\mathtt{b}$:

$$\mathtt{a}\,\mathtt{a}\,S\,\mathtt{b}\,\mathtt{b}$$

Agora a palavra cresce dos **dois lados ao mesmo tempo**,
e sempre na mesma quantidade.

**Nenhum estado de autômato representa isso,
porque não há como lembrar quantos faltam.**

Do autômato para a gramática

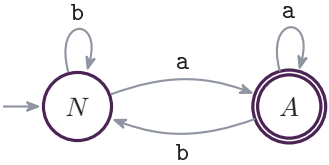
no autômato	**na gramática**
cada estado	vira um não terminal
flecha de X para Y lendo a	vira a regra $X \to$ a Y
estado final	ganha também a regra $X \to \varepsilon$
estado inicial	é por onde a derivação começa

Do autômato para a gramática

no autômato	**na gramática**
cada estado	vira um não terminal
flecha de X para Y lendo a	vira a regra $X \to \mathrm{a}\, Y$
estado final	ganha também a regra $X \to \varepsilon$
estado inicial	é por onde a derivação começa

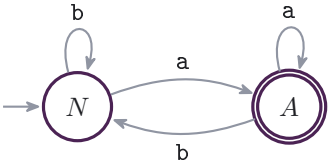
Uma regra por flecha. Derivar uma palavra passa a ser o mesmo que percorrer o autômato.

Aplicando ao autômato mínimo



$$N \to \mathtt{a}\ A \ \mid\ \mathtt{b}\ N$$
$$A \to \mathtt{a}\ A \ \mid\ \mathtt{b}\ N \ \mid\ \varepsilon$$

Aplicando ao autômato mínimo



$$N \to \mathtt{a}\ A \ \mid\ \mathtt{b}\ N$$

$$A \to \mathtt{a}\ A \ \mid\ \mathtt{b}\ N \ \mid\ \varepsilon$$

A barra vertical junta várias regras do mesmo não terminal numa linha.
O ε está em A porque A é final: pode-se encerrar ali.

Derivando `baa` nessa gramática

$$N$$

Derivando `baa` nessa gramática

$$N \Rightarrow \mathtt{b}N$$

no autômato: de N, lendo `b`, fiquei em N

Derivando baa nessa gramática

$$N \Rightarrow \mathtt{b}N \Rightarrow \mathtt{ba}A$$

de N, lendo a, fui para A

Derivando `baa` nessa gramática

$$N \Rightarrow \mathtt{b}N \Rightarrow \mathtt{ba}A \Rightarrow \mathtt{baa}A$$

de A, lendo a, fiquei em A

Derivando baa nessa gramática

$$N \Rightarrow \mathtt{b}N \Rightarrow \mathtt{ba}A \Rightarrow \mathtt{baa}A \Rightarrow \mathtt{baa}$$

A é final, então usei $A \to \varepsilon$ e parei

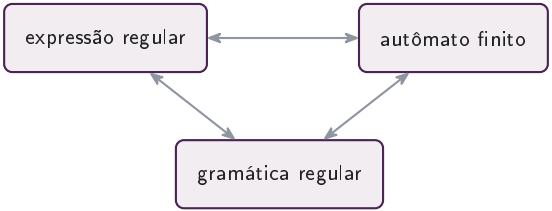
Derivando `baa` nessa gramática

$$N \Rightarrow \mathtt{b}N \Rightarrow \mathtt{ba}A \Rightarrow \mathtt{baa}A \Rightarrow \mathtt{baa}$$

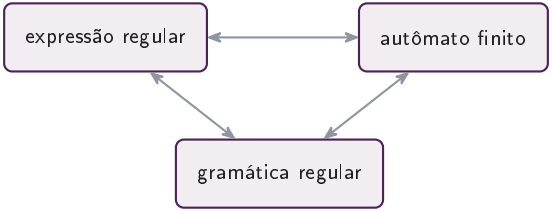
A é final, então usei $A \to \varepsilon$ e parei

Cada passo da derivação é um passo no autômato.
Conferi que as duas coincidem nas 255 palavras de até 7 símbolos.

Três notações, uma classe de linguagens

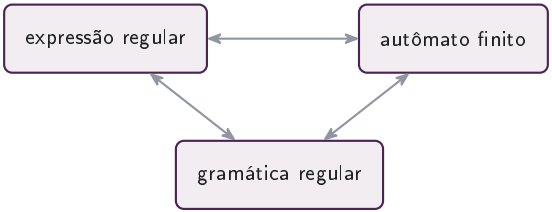


Três notações, uma classe de linguagens



expressão	para *escrever* o padrão rapidamente
autômato	para *executar*, virando código
gramática	para *gerar*, e para comparar com as classes maiores

Três notações, uma classe de linguagens



expressão	para *escrever* o padrão rapidamente
autômato	para *executar*, virando código
gramática	para *gerar*, e para comparar com as classes maiores

As três descrevem exatamente as mesmas linguagens.

8

Onde as regulares se encaixam

Quatro classes de linguagem

tipo	linguagens	reconhecidas por	quando vemos
3	regulares	autômato finito	esta unidade
2	livres de contexto	autômato de pilha	encontro 10
1	sensíveis ao contexto	máquina limitada	não veremos
0	recursivamente enumeráveis	máquina de Turing	encontro 20

Quatro classes de linguagem

tipo	linguagens	reconhecidas por	quando vemos
3	regulares	autômato finito	esta unidade
2	livres de contexto	autômato de pilha	encontro 10
1	sensíveis ao contexto	máquina limitada	não veremos
0	recursivamente enumeráveis	máquina de Turing	encontro 20

Chamam-se tipos de Chomsky, e cada um **contém** o de baixo:
toda linguagem regular também é livre de contexto.

O que muda de um tipo para o outro

Só uma coisa: **que formato as regras podem ter**.

tipo	**formato permitido**	**exemplo**
3	maiúscula só no fim	$N \to \mathrm{a}A$
2	maiúscula em qualquer lugar	$S \to (S)$

O que muda de um tipo para o outro

Só uma coisa: **que formato as regras podem ter**.

tipo	formato permitido	exemplo
3	maiúscula só no fim	$N \to \mathtt{a}A$
2	maiúscula em qualquer lugar	$S \to (S)$

Soltar essa única restrição já dá uma classe maior de linguagens.

Uma linguagem que autômato finito não reconhece

$$\mathtt{S} \rightarrow \mathtt{(}\ \mathtt{S}\ \mathtt{)} \mid \varepsilon$$

Uma linguagem que autômato finito não reconhece

$$\mathtt{S} \rightarrow \mathtt{(}\ \mathtt{S}\ \mathtt{)} \quad | \quad \varepsilon$$

Cada aplicação da primeira regra acrescenta um parêntese **de cada lado**.

$$\varepsilon, \quad \mathtt{()}, \quad \mathtt{(())}, \quad \mathtt{((()))}, \quad \ldots$$

Uma linguagem que autômato finito não reconhece

$$S \rightarrow (\ S \ ) \ \mid \ \varepsilon$$

Cada aplicação da primeira regra acrescenta um parêntese **de cada lado**.

$$\varepsilon, \quad (), \quad (()), \quad ((())), \quad \ldots$$

Nenhum autômato finito consegue reconhecer isso.

Uma linguagem que autômato finito não reconhece

$$S \to (\ S\ ) \mid \varepsilon$$

Cada aplicação da primeira regra acrescenta um parêntese **de cada lado**.

$$\varepsilon,\quad (),\quad (()),\quad ((())),\quad \ldots$$

Nenhum autômato finito consegue reconhecer isso.

A intuição: seria preciso *contar* quantos abriram, e a contagem não tem limite.
A demonstração é o encontro da semana que vem.

O que fica

- A mesma linguagem tem muitos autômatos, e um deles é o menor.

O que fica

- A mesma linguagem tem muitos autômatos, e um deles é o menor.
- Estado inalcançável: ninguém chega nele. Apague.

O que fica

- A mesma linguagem tem muitos autômatos, e um deles é o menor.
- Estado inalcançável: ninguém chega nele. Apague.
- Estados equivalentes: dali para a frente decidem igual. Junte.

O que fica

- A mesma linguagem tem muitos autômatos, e um deles é o menor.
- Estado inalcançável: ninguém chega nele. Apague.
- Estados equivalentes: dali para a frente decidem igual. Junte.
- O algoritmo separa finais de não finais e refina até nada mudar.

O que fica

- A mesma linguagem tem muitos autômatos, e um deles é o menor.
- Estado inalcançável: ninguém chega nele. Apague.
- Estados equivalentes: dali para a frente decidem igual. Junte.
- O algoritmo separa finais de não finais e refina até nada mudar.
- O mínimo é único, venha de que autômato vier.

O que fica

- A mesma linguagem tem muitos autômatos, e um deles é o menor.
- Estado inalcançável: ninguém chega nele. Apague.
- Estados equivalentes: dali para a frente decidem igual. Junte.
- O algoritmo separa finais de não finais e refina até nada mudar.
- O mínimo é único, venha de que autômato vier.
- Gramática é um conjunto de trocas: maiúscula é buraco, minúscula fica.

O que fica

- A mesma linguagem tem muitos autômatos, e um deles é o menor.
- Estado inalcançável: ninguém chega nele. Apague.
- Estados equivalentes: dali para a frente decidem igual. Junte.
- O algoritmo separa finais de não finais e refina até nada mudar.
- O mínimo é único, venha de que autômato vier.
- Gramática é um conjunto de trocas: maiúscula é buraco, minúscula fica.
- Regular é a gramática em que a maiúscula só aparece no fim.

O que fica

- A mesma linguagem tem muitos autômatos, e um deles é o menor.
- Estado inalcançável: ninguém chega nele. Apague.
- Estados equivalentes: dali para a frente decidem igual. Junte.
- O algoritmo separa finais de não finais e refina até nada mudar.
- O mínimo é único, venha de que autômato vier.
- Gramática é um conjunto de trocas: maiúscula é buraco, minúscula fica.
- Regular é a gramática em que a maiúscula só aparece no fim.
- E as regulares são o tipo 3, o menor da hierarquia de Chomsky.

O que fica

- A mesma linguagem tem muitos autômatos, e um deles é o menor.
- Estado inalcançável: ninguém chega nele. Apague.
- Estados equivalentes: dali para a frente decidem igual. Junte.
- O algoritmo separa finais de não finais e refina até nada mudar.
- O mínimo é único, venha de que autômato vier.
- Gramática é um conjunto de trocas: maiúscula é buraco, minúscula fica.
- Regular é a gramática em que a maiúscula só aparece no fim.
- E as regulares são o tipo 3, o menor da hierarquia de Chomsky.

Próximo encontro: por que os parênteses balanceados são impossíveis para um autômato finito.